WEEK 17.1 AI ASSISTED

M. VIKAS

2303A52190

BATCH 44 TASK1:

PROMPT: “Generate a Python script using Pandas to clean a customer dataset by handling missing values (age, income, city), removing duplicates, standardizing text fields, and encoding categorical variables like gender and city. Ensure the output is a cleaned dataframe ready for analysis.” CODE:

week 17.1.py > ...

```python
import pandas as pd

# Load dataset
data = pd.read_csv("shopping_trends.csv")

# Display original data
print("Original Data:")
print(data.head())

# ------------------------------
# 1. Handle Missing Values
# ------------------------------

# Fill missing numerical values with median
if 'Age' in data.columns:
    data['Age'].fillna(data['Age'].median(), inplace=True)

if 'Income' in data.columns:
    data['Income'].fillna(data['Income'].median(), inplace=True)

# Fill missing categorical values with mode
if 'City' in data.columns:
    data['City'].fillna(data['City'].mode()[0], inplace=True)

```

```python
if 'City' in data.columns:
    data['City'] = data['City'].str.lower().str.strip()

if 'Gender' in data.columns:
    data['Gender'] = data['Gender'].str.lower().str.strip()

# ------------------------------
# 4. Encode Categorical Variables
# ------------------------------

# One-hot encoding for City
if 'City' in data.columns:
    data = pd.get_dummies(data, columns=['City'], drop_first=True)

# Label encoding for Gender
if 'Gender' in data.columns:
    data['Gender'] = data['Gender'].map({'male': 0, 'female': 1})

# ------------------------------
# Final Output
# ------------------------------
print("\nCleaned Data:")
print(data.head())

```

```python

# Save cleaned dataset
data.to_csv("cleaned_shopping_trends.csv", index=False)
```

Output:

```
Original Data:
   Customer ID  Age  ... Preferred Payment Method Frequency of Purchases
0            1   55  ...                    Venmo            Fortnightly
1            2   19  ...                     Cash            Fortnightly
2            3   50  ...              Credit Card                 Weekly
3            4   21  ...                   PayPal                 Weekly
4            5   45  ...                   PayPal               Annually

[5 rows x 19 columns]

Cleaned Data:
   Customer ID  Age  ...  Preferred Payment Method Frequency of Purchases
0            1   55  ...                     Venmo            Fortnightly
1            2   19  ...                      Cash            Fortnightly
2            3   50  ...               Credit Card                 Weekly
3            4   21  ...                    PayPal                 Weekly
4            5   45  ...                    PayPal               Annually

[5 rows x 19 columns]
PS C:\Users\vikas\Downloads\AI Assist Coding>
```

Summary: This script cleans a customer dataset by handling missing values, removing duplicates, and standardizing text fields like city and gender. It also converts categorical data

into numerical format using encoding techniques. The final cleaned dataset is saved and ready for further analysis or machine learning tasks.

Task 2:

Prompt: "Generate a Python script using Pandas to preprocess sales data by converting date columns to datetime, creating time-based features (month, year, day of week), normalizing the sales column, and detecting and handling outliers using the IQR method. Ensure the output dataset is ready for analysis."

Code:

```python
import pandas as pd
import numpy as np

# Load dataset
data = pd.read_csv("sales_data.csv")

# Display original data
print("Original Data:")
print(data.head())

# -------------------------------
# 1. Convert Date Column
# -------------------------------
if 'Date' in data.columns:
    data['Date'] = pd.to_datetime(data['Date'], errors='coerce')

# -------------------------------
# 2. Create Time-Based Features
# -------------------------------
if 'Date' in data.columns:
    data['Year'] = data['Date'].dt.year
    data['Month'] = data['Date'].dt.month
    data['DayOfWeek'] = data['Date'].dt.day_name()
```

```python
# ------------------------------
if 'Sales' in data.columns:
    data['Sales_Normalized'] = (data['Sales'] - data['Sales'].min()) / (data['Sa

# ------------------------------
# 4. Detect and Handle Outliers (IQR Method)
# ------------------------------
if 'Sales' in data.columns:
    Q1 = data['Sales'].quantile(0.25)
    Q3 = data['Sales'].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Replace outliers with median
    median_value = data['Sales'].median()
    data['Sales'] = np.where((data['Sales'] < lower_bound) | (data['Sales'] > up
                             median_value,
                             data['Sales'])

# ------------------------------
# Final Output
# ------------------------------
print("\nProcessed Data:")
print(data.head())

# Save processed dataset
data.to_csv("processed_sales_data.csv", index=False)
```

Output:

```
[5 rows x 19 columns]
Original Data:
   Product_ID   Sale_Date Sales_Rep  ... Payment_Method  Sales_Channel  Region_and_Sales_Rep
0        1052  2023-02-03       Bob  ...           Cash         Online             North-Bob
1        1093  2023-04-21       Bob  ...           Cash         Retail              West-Bob
2        1015  2023-09-21     David  ...  Bank Transfer         Retail           South-David
3        1072  2023-08-24       Bob  ...    Credit Card         Retail             South-Bob
4        1061  2023-03-24   Charlie  ...    Credit Card         Online          East-Charlie

[5 rows x 14 columns]

Processed Data:
   Product_ID   Sale_Date Sales_Rep  ... Payment_Method  Sales_Channel  Region_and_Sales_Rep
0        1052  2023-02-03       Bob  ...           Cash         Online             North-Bob
1        1093  2023-04-21       Bob  ...           Cash         Retail              West-Bob
2        1015  2023-09-21     David  ...  Bank Transfer         Retail           South-David
3        1072  2023-08-24       Bob  ...    Credit Card         Retail             South-Bob
4        1061  2023-03-24   Charlie  ...    Credit Card         Online          East-Charlie

[5 rows x 14 columns]
```

```
[5 rows x 19 columns]
Original Data:
   Product_ID   Sale_Date  ... Sales_Channel Region_and_Sales_Rep
0        1052  2023-02-03  ...        Online            North-Bob
1        1093  2023-04-21  ...        Retail             West-Bob
2        1015  2023-09-21  ...        Retail          South-David
3        1072  2023-08-24  ...        Retail            South-Bob
4        1061  2023-03-24  ...        Online         East-Charlie

[5 rows x 14 columns]

Processed Data:
   Product_ID   Sale_Date  ... Sales_Channel Region_and_Sales_Rep
0        1052  2023-02-03  ...        Online            North-Bob
1        1093  2023-04-21  ...        Retail             West-Bob
2        1015  2023-09-21  ...        Retail          South-David
3        1072  2023-08-24  ...        Retail            South-Bob
4        1061  2023-03-24  ...        Online         East-Charlie
```

Summary: This script preprocesses sales data by converting date columns into datetime format and extracting useful time-based features like year, month, and day of the week. It normalizes the sales values for better analysis and applies the IQR method to detect and handle outliers. The final dataset is clean, structured, and ready for analytics or machine learning.

TASK 3:

Prompt: "Generate a Python script using Pandas and Scikit-learn to preprocess a healthcare dataset by handling missing values in blood pressure and cholesterol, encoding categorical variables, scaling numerical features, and splitting the dataset into training and testing sets for machine learning." Code:

```python
#----------------------------task 3-------------------------------#
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.model_selection import train_test_split

# Load dataset
data = pd.read_csv("dirty_v3_path.csv")

# Display original data
print("Original Data:")
print(data.head())

# --------------------------------------
# 1. Handle Missing Values
# --------------------------------------

# Fill numerical columns with median
if 'blood_pressure' in data.columns:
    data['blood_pressure'].fillna(data['blood_pressure'].median(), inplace=True)

if 'cholesterol' in data.columns:
    data['cholesterol'].fillna(data['cholesterol'].median(), inplace=True)
```

```python
label_encoders = {}

for col in data.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col].astype(str))
    label_encoders[col] = le

# ---------------------------------------
# 3. Scale Numerical Features
# ---------------------------------------
scaler = MinMaxScaler()

num_cols = data.select_dtypes(include=['int64', 'float64']).columns
data[num_cols] = scaler.fit_transform(data[num_cols])

# ---------------------------------------
# 4. Split Dataset into Train & Test
# ---------------------------------------

# Assuming 'target' is the label column (change if needed)
target_column = data.columns[-1]  # last column as target

X = data.drop(target_column, axis=1)
y = data[target_column]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```python
)

# ---------------------------------------
# Final Output
# ---------------------------------------
print("\nPreprocessed Data:")
print(data.head())

print("\nTraining Set Shape:", X_train.shape)
print("Testing Set Shape:", X_test.shape)
```

```
[5 rows x 14 columns]
Original Data:
    Age  Gender Medical Condition  ...  Sleep Hours  random_notes   noise_col
0  46.0    Male          Diabetes  ...         6.05         lorem -137.057211
1  22.0    Male           Healthy  ...         7.72         ipsum  -11.230610
2  50.0     NaN            Asthma  ...         4.82         ipsum   98.331195
3  57.0     NaN           Obesity  ...         5.33         lorem   44.187175
4  66.0  Female      Hypertension  ...         6.64         lorem   44.831426

[5 rows x 20 columns]

Preprocessed Data:
        Age  Gender  Medical Condition  ...  Sleep Hours  random_notes  noise_col
0  0.455696       1                  3  ...     0.509132             3   0.312605
1  0.151899       1                  4  ...     0.699772             2   0.455579
2  0.506329       2                  1  ...     0.368721             2   0.580072
3  0.594937       2                  6  ...     0.426941             3   0.518549
4  0.708861       0                  5  ...     0.576484             3   0.519281

[5 rows x 20 columns]

Training Set Shape: (24000, 19)
Testing Set Shape: (6000, 19)
```

Summary:

This script prepares a healthcare dataset by filling missing values in key medical fields like blood pressure and cholesterol. It encodes categorical features, scales numerical data for consistency, and splits the dataset into training and testing sets. The result is a clean and structu;red dataset ready for machine learning models.

TASK 4:

PROMPT: “Generate a Python script using Pandas and Scikit-learn to preprocess an employee salary dataset by handling missing values, removing salary outliers, standardizing department names, and applying label encoding to job location and department columns.”
Code:

```python
#-----------------task 4-------------------------------#
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load dataset
data = pd.read_csv("Employee_Salary_Dataset.csv")

# Display original data
print("Original Data:")
print(data.head())

# ---------------------------------------
# 1. Handle Missing Values
# ---------------------------------------

# Fill numerical columns with median
num_cols = data.select_dtypes(include=['int64', 'float64']).columns
for col in num_cols:
    data[col].fillna(data[col].median(), inplace=True)
```

```python
cat_cols = data.select_dtypes(include=['object']).columns
for col in cat_cols:
    data[col].fillna(data[col].mode()[0], inplace=True)

# --------------------------------------
# 2. Detect & Remove Salary Outliers (IQR)
# --------------------------------------

# Adjust column name if different
salary_col = 'salary' if 'salary' in data.columns else data.columns[-1]

Q1 = data[salary_col].quantile(0.25)
Q3 = data[salary_col].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

data = data[(data[salary_col] >= lower_bound) & (data[salary_col] <= upper_bound

```

```python
if 'department' in data.columns:
    data['department'] = data['department'].str.lower().str.strip()

# --------------------------------------
# 4. Label Encoding
# --------------------------------------
le = LabelEncoder()

for col in ['job_location', 'department']:
    if col in data.columns:
        data[col] = le.fit_transform(data[col])

# --------------------------------------
# Final Output
# --------------------------------------
print("\nProcessed Data:")
print(data.head())

# Save cleaned dataset
data.to_csv("processed_employee_salary.csv", index=False)
```

Output:

```
Original Data:
   ID  Experience_Years  Age  Gender  Salary
0   1                 5   28  Female  250000
1   2                 1   21    Male   50000
2   3                 3   23  Female  170000
3   4                 2   22    Male   25000
4   5                 1   17    Male   10000

Processed Data:
   ID  Experience_Years  Age  Gender  Salary
0   1                 5   28  Female  250000
1   2                 1   21    Male   50000
2   3                 3   23  Female  170000
3   4                 2   22    Male   25000
4   5                 1   17    Male   10000
PS C:\Users\vikas\Downloads\AI Assist Coding>
```

Summary:

This script preprocesses employee salary data by handling missing values and removing salary outliers using the IQR method. It standardizes department names for consistency and encodes categorical fields like job location and department. The final dataset is clean and structured for HR analytics and decision-making.